

Rapport final - Projet NoSQL : Libreville Eats

Module : Bases de données NoSQL - Master I

Groupe : Groupe 3

Membres : Serge CJ (à compléter avec les autres membres du groupe)

1. Introduction

Libreville Eats est une application de livraison de repas à domicile pour la ville de Libreville (Gabon). Le projet couvre l'intégralité d'un backend de production : authentification, gestion des restaurants et de leurs menus, prise de commande, suivi de livraison en temps réel (GPS), et un tableau de bord pour chacun des quatre rôles de la plateforme (client, restaurateur, livreur, administrateur).

Au-delà de l'exigence du module (modélisation NoSQL, scripts CRUD/requêtes/agrégations, indexation, analyse explain()), ce dépôt va plus loin en livrant une API REST Node.js/Express/MongoDB/Redis fonctionnelle ainsi qu'un frontend Next.js de démonstration, afin d'ancrer chaque concept NoSQL étudié dans un cas d'usage réel plutôt que dans un script isolé.

2. Contexte métier

Le marché de la livraison de repas à Libreville présente des contraintes spécifiques :

- Connectivité variable : les temps de réponse de l'API doivent rester bas même sur réseau mobile dégradé - d'où l'usage de Redis en cache devant MongoDB.
- Suivi temps réel : un client doit voir la position de son livreur évoluer en direct (Socket.io + relevés GPS horodatés).
- Diversité des acteurs : un même utilisateur authentifié peut être client, restaurateur ou livreur - d'où un schéma users unique avec un champ role, plutôt que des collections séparées par type d'acteur.
- Données réalistes localisées : adresses, quartiers (Nombakélé, Batavéa, Akébé...), plats (Poulet Nyembwe, Odika...), numéros de téléphone et devise (FCFA) gabonais, pour que les jeux de test et les démonstrations soient représentatifs.

Ces contraintes ont directement guidé les choix de modélisation détaillés en section 3.

3. Modélisation NoSQL

Quatre collections principales : users, restaurants, commandes, deliverers. Le détail complet (schémas JSON, choix embedding/referencing, justifications) se trouve dans conception/modele-donnees.md et le schéma visuel dans conception/schema.png. Synthèse des arbitrages :

Donnée	Choix	Raison
Menus & plats d'un restaurant	Embedding	Toujours lus ensemble, taille bornée
`items[]` d'une commande	Embedding (snapshot)	Le prix/nom au moment de l'achat ne doit pas changer
Historique de statuts et tracé GPS	Embedding	Audit trail et time-series locaux, toujours
`customer_id` / `restaurant_id` / `deliverer_id`	Referencing	Entités à cycle de vie propre et cardinalité 1
`owner_id` d'un restaurant	Referencing	Un propriétaire peut posséder plusieurs restaurants

4. Implémentation

Le seed de données réaliste (contexte gabonais) est fourni à deux niveaux :

- data/seed.js : version mongosh pure (insertMany()), chargeable directement dans mongosh sans Node.js/Mongoose, pour la soutenance.
- scripts/seed/index.js + scripts/data/gabon-data.js : générateur utilisé par l'application réelle, exécuté via npm run seed.

Les deux scripts produisent le même jeu de données réel : 30 vraies enseignes de Libreville/Akanda/Owendo (KFC, PAUL, Blé Doré, La Braise, Bantu...) avec leurs vrais plats et horaires, 90 comptes utilisateurs (30 clients, 30 restaurateurs - un par restaurant -, 30 livreurs, tous avec des noms gabonais et le mot de passe TestPass123 en clair pour la démonstration), 30 profils livreurs, et un échantillon de commandes cohérentes. Pour la présentation, 18 des 30 restaurants sont volontairement marqués fermés (isOpen: false) et 1 plat sur 10 par restaurant (10%) est marqué en rupture de stock (isAvailable: false), afin de pouvoir démontrer en direct le filtrage sur ces champs.

Les opérations CRUD de base (insertOne, insertMany, find, updateOne, updateMany, deleteOne) sont démontrées dans scripts/01-crud.js, directement chargeable dans mongosh.

5. Requêtes et agrégations

- scripts/02-requetes.js couvre les filtres de comparaison (\$gt, \$lt), l'opérateur \$in, les expressions régulières, le tri (sort) et la pagination (skip/limit).
- scripts/03-agregations.js couvre trois pipelines \$match / \$group / \$sort / \$lookup : statistiques de commandes par restaurant, performance des livreurs, et analyse des revenus par moyen de paiement et par restaurant. Ces pipelines reprennent la logique exacte utilisée en production par l'application (scripts/aggregations/*.js, exécutée via Mongoose et exposée par npm run aggregate:all), réécrite ici en db.collection.aggregate() pur pour être exécutable directement dans mongosh.

6. Indexation

scripts/04-index.js crée l'ensemble des index de l'application :

- Unique : users.email (authentification).
- Géospatiaux (2dsphere) : users.addresses.coordinates, restaurants.address.coordinates, deliverers.currentLocation, commandes.deliveryTracking.coordinates - pour les recherches de proximité et le suivi GPS.
- Texte pondéré : restaurants.name / menus.name / menus.dishes.name / menus.dishes.description - pour la recherche plein-texte.
- Composés : commandes (status, restaurant_id) - index critique pour le tableau de bord vendeur -, commandes (customer_id, createdAt) pour l'historique client, commandes (deliverer_id, status) pour le suivi des livraisons, restaurants (address.district, isOpen), et deliverers (isAvailable, isActive).

Le script mesure lui-même, avant et après création de l'index status_1_restaurant_id_1, le plan d'exécution de la requête de référence (commandes PREPARING d'un restaurant) et affiche la bascule COLLSCAN -> IXSCAN.

7. Analyse des performances

Détail complet (captures d'écran mongosh incluses) dans explain/explain-avant-apres.pdf, mesuré en conditions réelles sur la base libreville_eats (45 commandes) :

Métrique	Sans index (COLLSCAN)	Avec index (IXSCAN)
Stage gagnant	COLLSCAN	FETCH <- IXSCAN
Documents examinés	45	1
Documents retournés	1	1
Ratio examinés / retournés	45	1

Sans index, le coût d'une requête croît linéairement avec la taille de la collection, quelle que soit la sélectivité du filtre. Avec l'index composé (status, restaurant_id), le coût ne dépend plus que du nombre de documents réellement retournés - un comportement reproductible à tout moment via `load("scripts/04-index.js")`, preuve que la bascule d'optimiseur est indépendante du volume, seul l'écart de coût absolu se creusant avec la taille réelle des données (en production, avec des milliers de commandes, le même ratio 45:1 passerait à plusieurs milliers:1).

8. Conclusion

Le projet démontre, sur un cas d'usage réel de livraison de repas, l'ensemble des compétences NoSQL attendues : modélisation par arbitrage embedding/referencing justifié, opérations CRUD, requêtes avancées (filtres, regex, tri, pagination), pipelines d'agrégation multi-étapes avec jointures (\$lookup), et stratégie d'indexation mesurée par `explain('executionStats')`.

Au-delà du socle obligatoire, l'implémentation complète (API Express/Mongoose, cache Redis, authentification JWT, suivi temps réel Socket.io, frontend Next.js) - détaillée dans le README racine du dépôt et le dossier `api/` - montre comment ces choix de modélisation se traduisent dans une application en conditions réelles : machine à états des commandes (`src/utls/orderStateMachine.js`), invalidation de cache cohérente avec les clés MongoDB (`src/services/cacheService.js`), et index géospatiaux exploités par les fonctionnalités de proximité (restaurants à proximité, livreurs disponibles).

Pistes d'amélioration

- Partitionnement (sharding) de commandes par `restaurant_id` si le volume dépassait l'échelle d'un seul nœud.
- Time-series collection MongoDB native pour `deliveryTracking` au-delà d'un certain volume de points GPS par commande.
- Vues matérialisées (collection dédiée mise à jour par triggers) pour les statistiques agrégées les plus consultées, afin d'éviter de recalculer les pipelines à chaque requête du tableau de bord.